

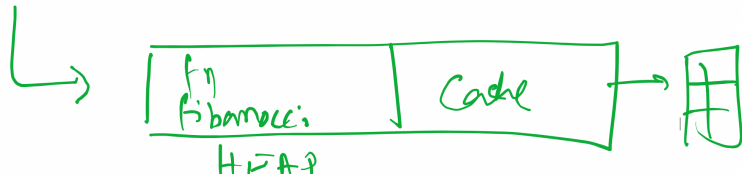
> memFib.prototype

```
<  ▾ {} i
  ▾ constructor: f (args)
    length: 1
    name: ""
    ▸ prototype: {}
      arguments: null
      caller: null
      [[FunctionLocation]]: memo.html:14
    ▸ [[Prototype]]: f ()
  ▾ [[Scopes]]: Scopes[3]
    ▾ 0: Closure (memoize)
      ▸ cache: {40: 102334155}
      ▸ fn: f fibanocci(no)
      ▸ [[Prototype]]: Object
      ▸ 1: Script {memFib: f}
      ▸ 2: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}
    ...
```

```
function memoize(fn) {
  var cache = {};
  return function(args) {
    if(cache[args]) {
      return cache[args];
    } else {
      let result = fn(args);
      cache[args] = result;
      return result;
    }
  }
}
```

```
function fibanocci(no) {
  if(no == 0 || no == 1) {
    return no;
  } else {
    return fibanocci(no - 1) + fibanocci(no - 2);
  }
}
```

let memFib = memoize(fibanocci);



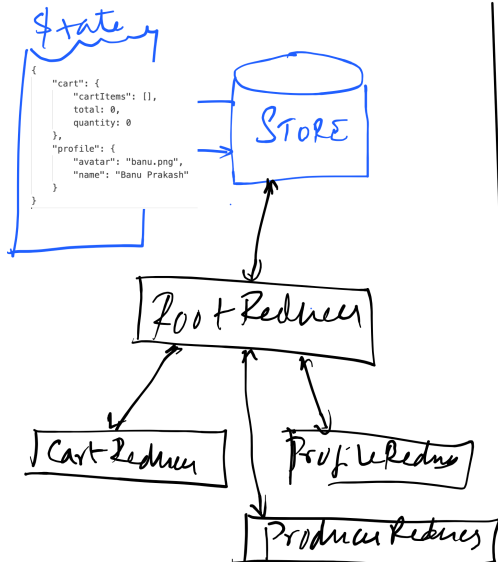
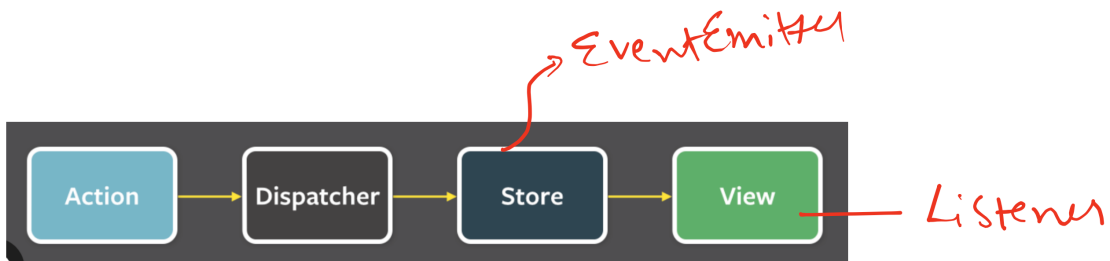
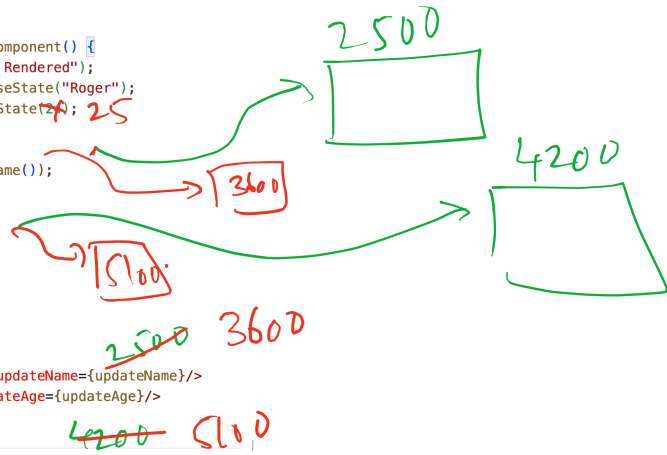
Re-rendered

```
export default function ParentComponent() {
  console.log ("ParentComponent Rendered");
  let [name, setName] = React.useState("Roger");
  let [age, setAge] = React.useState(25);

  function updateName() {
    setName(faker.person.firstName());
  }

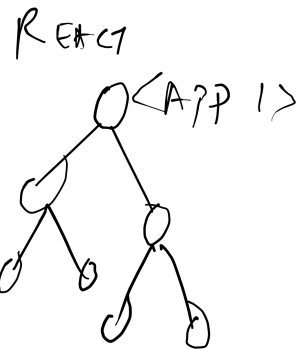
  function updateAge() {
    setAge(age + 1);
  }

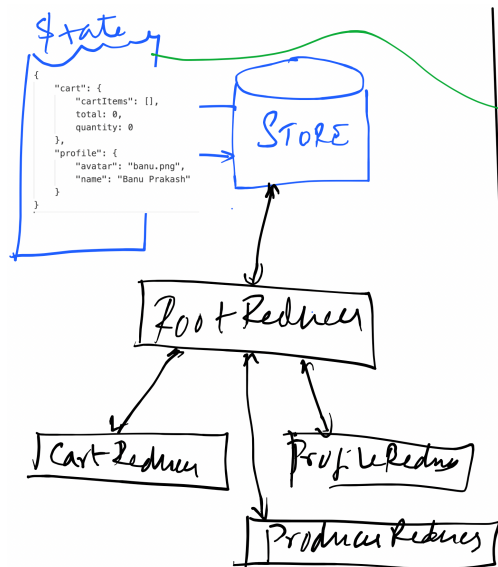
  return (
    <div>
      <h1>ParentComponent</h1>
      <NameComponent name={name} updateName={updateName}/>
      <AgeComponent age={age} updateAge={updateAge}/>
    </div>
  )
}
```



React-Redux

```
connect(
  mapStateToProps,
  mapDispatchToProps
)(App)
```





React-Redux

```

// state is from redux store
function mapStateToProps(state) {
  return {
    "items": state.cart.cartItems,
    "pic": state.profile.avatar
  }
}

```

```

connect(
  mapStateToProps,
  mapDispatchToProps
)(App)

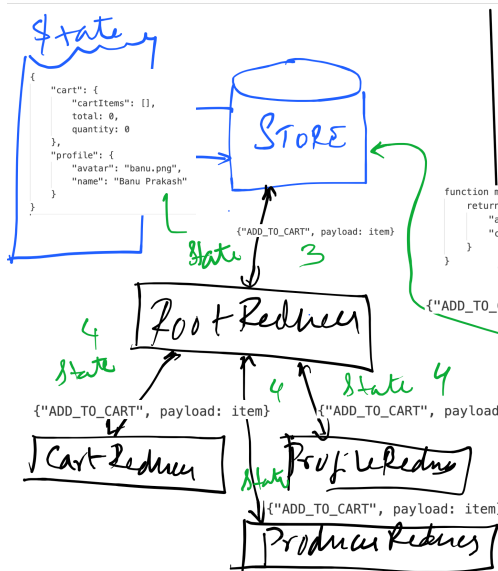
```

React

```

<App />
<img src={props.pic} />
{
  props.items.map(...)
}

```



React-Redux

```

function mapDispatchToProps(dispatch) {
  return {
    "addToCart": item => dispatch({ "ADD_TO_CART": item, payload: item }),
    "clear": () => dispatch({ "CLEAR_CART": item })
  }
}

```

```

{"ADD_TO_CART", payload: item}

```

```

connect(
  mapStateToProps,
  mapDispatchToProps
)(App)

```

React

```

<App />
props.addToCart({..})
props.clearCart();

```

